

IntSeqBERT: Learning Arithmetic Structure in OEIS via Modulo-Spectrum Embeddings

Kazuhisa Nakasho¹

Iwate Prefectural University, Takizawa, Iwate, Japan
nakasho_k@iwate-pu.ac.jp

Abstract. Integer sequences in the OEIS span values from single-digit constants to astronomical factorials and exponentials, making prediction challenging for standard tokenised models that cannot handle out-of-vocabulary values or exploit periodic arithmetic structure. We present **IntSeqBERT**, a dual-stream Transformer encoder for masked integer-sequence modelling on OEIS. Each sequence element is encoded along two complementary axes: a continuous log-scale magnitude embedding and sin/cos modulo embeddings for 100 residues (moduli 2–101), fused via FiLM. Three prediction heads (magnitude regression, sign classification, and modulo prediction for 100 moduli) are trained jointly on 274,705 OEIS sequences. At the Large scale (91.5M parameters), IntSeqBERT achieves 95.85% magnitude accuracy and 50.38% Mean Modulo Accuracy (MMA) on the test set, outperforming a standard tokenised Transformer baseline by +8.9pt and +4.5pt, respectively. An ablation removing the modulo stream confirms it accounts for +15.2pt of the MMA gain and contributes an additional +6.2pt to magnitude accuracy. A probabilistic Chinese Remainder Theorem (CRT)-based Solver converts the model’s predictions into concrete integers, yielding a 7.4-fold improvement in next-term prediction over the tokenised-Transformer baseline (Top-1: 19.09% vs. 2.59%). Modulo spectrum analysis reveals a strong negative correlation between Normalised Information Gain (NIG) and Euler’s totient ratio $\varphi(m)/m$ ($r = -0.851$, $p < 10^{-28}$), providing empirical evidence that composite moduli capture OEIS arithmetic structure more efficiently via CRT aggregation.

Keywords: Integer sequences · OEIS · Masked sequence modelling · Modular arithmetic · Transformer · FiLM

1 Introduction

The On-Line Encyclopedia of Integer Sequences (OEIS) [15], as of January 2026, catalogues 391,710 entries spanning combinatorics, number theory, algebra, and many other branches of mathematics, making it the *de facto* standard reference for integer sequences. Each entry associates a finite integer sequence with its mathematical definition, rendering the OEIS a uniquely machine-readable corpus of mathematical knowledge.

The long-term goal of this research is an AI system that discovers hidden number-theoretic similarities between integer sequences arising in different mathematical fields or from different algorithms, and thereby supports the generation of mathematical conjectures—a goal in the same spirit as recent AI-driven discovery systems such as AlphaProof [9], the Ramanujan Machine [12], and FunSearch [14] (Section 2.1). A prerequisite for this goal is a machine representation that captures the number-theoretic structure of OEIS sequences. This paper takes a first step: we propose such a representation and measure, through a masked-prediction task, how much arithmetic structure it actually captures.

The task we formalise is *masked sequence modelling*: a random subset of positions in a sequence is masked and the model is trained to predict each masked value from its surrounding context. This task directly probes how well a model has internalised the arithmetic and combinatorial laws governing integer sequences; next-term prediction is treated as a special case and evaluated via a dedicated Solver component. The prior benchmark FACT [1] defines and evaluates five tasks including unmasking, establishing what a plain Transformer can achieve, but it faces fundamental limitations in handling large integer values outside its fixed token vocabulary and in learning multiplicative structure.

The principal difficulty of this task is the extreme heterogeneity of integer sequences. Values range from single-digit constants to astronomically large factorials and exponential sequences, with differences of tens of orders of magnitude within a single training corpus. At the same time, many sequences obey *residue constraints*—parity, combinatorial patterns, or periodic remainders—that are independent of the magnitude of individual values. The standard tokenisation approach of assigning each integer a discrete vocabulary token is fundamentally ill-suited to this setting: it cannot represent integers outside its fixed token vocabulary, embeds arithmetic structure in opaque token IDs, and breaks down in scale for large numbers.

We propose **IntSeqBERT**, a Transformer encoder pre-trained on the OEIS corpus via masked sequence modelling, which overcomes the above challenges through a *dual-stream* input representation. Rather than tokenising integers, IntSeqBERT encodes each element along two complementary axes:

- **Magnitude stream**: a continuous logscale embedding of the absolute value, capturing growth behaviour and scale.
- **Modulo stream**: sin/cos embeddings for 100 residues modulo 2 through 101, capturing periodicity and number-theoretic structure.

The two streams are fused via FiLM (Feature-wise Linear Modulation) [11]. During training, three predictors are jointly optimised in a multi-task objective: magnitude regression, sign classification, and modulo prediction for 100 moduli. Concrete integer values are recovered from the masked-position predictions (magnitude, sign, residue distributions) using a probabilistic Chinese Remainder Theorem (CRT)-based **Solver** (Section 3.5).

This combination of the modulo-spectrum representation and the CRT solver is a *neural-symbolic* design: residue arithmetic and the Chinese Remainder Theorem are built into the model and the decoding procedure as explicit number-

theoretic structure, rather than left for the network to discover end-to-end. Our contribution is therefore not the prediction accuracy itself—predicting integer sequences via machine learning is a long-standing task, e.g. the Kaggle Integer Sequence Learning competition [10]—but the representation that explicitly encodes number-theoretic structure, together with quantitative evidence that this representation captures the structure of OEIS sequences. At the same time, our results document how difficult direct neural prediction of OEIS terms remains even with this structure built in, providing motivation for hybrid symbolic–neural approaches (Section 7).

We evaluate IntSeqBERT against two baselines on 274,705 OEIS sequences across three model sizes (Small: 6.4M, Middle: 29.0M, Large: 91.5M parameters), all on a single GeForce RTX 3070 Ti (8 GB VRAM). The implementation, preprocessing scripts, and evaluation code are publicly available.¹

Contributions.

1. *IntSeqBERT architecture*: a dual-stream Transformer fusing magnitude and modular-arithmetic embeddings via FiLM, jointly trained on OEIS with magnitude regression, sign classification, and 100-dimensional modulo prediction. At the Large scale, IntSeqBERT achieves **95.85%** magnitude accuracy and **50.38%** MMA, surpassing Vanilla by +8.9 pt and +4.5 pt, respectively. The dual-stream representation yields a **7.4-fold** improvement in Solver-based next-term prediction (Top-1: 19.09% vs. 2.59%).
2. *Number-theoretic finding*: modulo spectrum analysis reveals that NIG is strongly negatively correlated with $\varphi(m)/m$ (where φ denotes Euler’s totient function; Pearson correlation of $r = -0.851$ ($p < 10^{-28}$)), providing quantitative evidence that composite moduli aggregate arithmetic structure more efficiently via CRT.
3. *Scaling behaviour*: modulo accuracy and Solver accuracy improve more steeply with model size than magnitude accuracy, suggesting arithmetic reasoning benefits disproportionately from increased capacity.

Paper organisation. Sections 2–4 review related work, present the architecture, and describe the experimental setup. Section 5 reports results, Section 6 provides analyses, and Section 7 concludes.

2 Background and Related Work

2.1 AI for Mathematics

Throughout mathematics, numerical pattern observation has seeded important conjectures—from Monstrous Moonshine [4] and the BSD conjecture [2] to Wiles’s proof of the Shimura–Taniyama conjecture for semistable elliptic curves, which resolved Fermat’s Last Theorem [19], and Candelas et al.’s mirror-symmetry prediction of 317,206,375 rational curves [3]. Recent AI systems including AlphaGeometry [16], AlphaProof [9], the Ramanujan Machine [12], and FunSearch [14]

¹ <https://github.com/aabaa/IntSeqBERT>

demonstrate the viability of AI-driven mathematical discovery. On the formalisation front, Urban’s large-scale autoformalization work [17] shows that natural-language mathematics can be automatically translated into machine-verifiable form. Yet extracting arithmetic laws from integer sequences at scale remains largely unexplored. This work aims to establish representational foundations for that capability.

2.2 AI Research on OEIS

Program-synthesis approaches to OEIS include Alien Coding [6], which searches for the shortest reproducing code; QSynt [7], which synthesises programs for 43,516 sequences via self-learning tree search; and Learning Conjecturing [8], which automatically proves 5,565 arithmetic problems. FACT [1] (Belčák et al., NeurIPS 2022) constructs a comprehensive OEIS benchmark covering five tasks in order of difficulty—classification, similarity, next sequence-part prediction, continuation, and *unmasking* (identified as the hardest). Our Vanilla baseline corresponds to FACT’s plain Transformer; IntSeqBERT extends it with modulo feature engineering and FiLM fusion to overcome FACT’s limitations with large integers and multiplicative structure. The Solver-based next-term prediction evaluated in Section 5.4 corresponds to FACT’s continuation task.

2.3 Modulo Spectra as Arithmetic Feature Engineering

Deep networks exhibit a well-known *spectral bias* [13]: they learn low-frequency functions before high-frequency ones. In the context of integer sequences, this is particularly problematic for multiplicatively growing sequences (e.g., n^2 , $n!$), where learning the growth law from token sequences alone requires many layers. This work mitigates the problem through feature engineering: because the results of multiplication appear naturally in modular residues, supplying the modulo spectrum as explicit input features reduces the network depth required to “rediscover” multiplicative structure.

More precisely, most OEIS sequences are generated by finite algorithms combining addition, multiplication, and modular operations. Since modular arithmetic is a ring homomorphism over $\mathbb{Z}$, the residue sequence $(x_i \bmod m)$ faithfully preserves additive and multiplicative structure as a compact representation, regardless of the magnitudes involved. Fermat’s little theorem implies that $(a^n \bmod p)$ is periodic with period dividing $p - 1$ for prime p with $\gcd(a, p) = 1$. Gauss’s law of quadratic reciprocity [5] and its generalisations further link residue information across different primes. Composite moduli simultaneously retain information from all prime power factors via CRT, motivating the use of both prime and composite moduli in our spectrum. Our modulo features cover $m \in \{2, 3, \dots, 101\}$ (100 moduli), capturing power-residue structure broadly across primes and composites.

2.4 Positioning and Neural Representations

Tokenisation (the LLM mainstream) cannot handle out-of-vocabulary values and buries arithmetic structure in opaque token IDs. Log-scale continuous embeddings improve scale invariance but miss arithmetic periodicity. We instead repurpose sinusoidal embeddings [18] from positions to *modular residues of values*, and apply FiLM [11] to modulate the magnitude stream via the modulo stream.

In contrast to the Gauthier and Urban group’s program-synthesis line [6,7,8], which explicitly describes each sequence’s generation rule, this work uses *representation learning* to acquire shared arithmetic structure across the OEIS corpus. We view the two approaches as complementary: program synthesis seeks a formula or program that generates a given sequence, whereas our representations aim to judge whether two sequences are number-theoretically similar even when they are generated in entirely different ways. Pre-trained IntSeqBERT embeddings could in turn serve as a search heuristic for program synthesis, or as an index for retrieving similar sequences. Where FACT [1] identified the limits of a plain tokenised Transformer, IntSeqBERT overcomes them via modulo-spectrum feature engineering and FiLM fusion.

3 IntSeqBERT

3.1 Problem Formulation

Let $\mathbf{x} = (x_1, x_2, \dots, x_L)$ with $x_i \in \mathbb{Z}$ and $L \leq 128$ be a finite prefix of an OEIS sequence. We adopt *masked sequence modelling*: a random subset of positions is masked and the model is trained to predict each masked value. Specifically, for each masked position i , three quantities are predicted:

1. **Magnitude:** $v_i = \begin{cases} 0 & (x_i = 0), \\ 1 + \log_{10} |x_i| & (x_i \neq 0) \end{cases} \in \mathbb{R}_{\geq 0}$
2. **Sign:** $s_i \in \{+, -, 0\}$ (3-class label)
3. **Residues:** $r_i^{(m)} = x_i \bmod m$ for each $m \in \{2, 3, \dots, 101\}$ (100 independent classification targets)

This decomposition separates magnitude, sign, and periodic arithmetic structure into complementary supervision signals.

3.2 Dual-Stream Representation

Two feature vectors are computed for each element x_i prior to learnable embeddings.

Magnitude features $\mathbf{f}_i^{\text{mag}} \in \mathbb{R}^4$:

$$\mathbf{f}_i^{\text{mag}} = [v_i, \mathbf{1}[x_i > 0], \mathbf{1}[x_i < 0], \mathbf{1}[x_i = 0]].$$

The last three components are a one-hot sign representation. For astronomically large integers exceeding the `float64` range, $|x_i|$ is replaced by its decimal digit count.

Modulo features $\mathbf{f}_i^{\text{mod}} \in \mathbb{R}^{200}$: For each modulus $m \in \{2, \dots, 101\}$, let $r = x_i \bmod m$, and embed the residue as a point on the unit circle:

$$\phi_m(r) = \left[\sin\left(\frac{2\pi r}{m}\right), \cos\left(\frac{2\pi r}{m}\right) \right] \in \mathbb{R}^2.$$

Concatenating all 100 moduli gives $\mathbf{f}_i^{\text{mod}} \in \mathbb{R}^{200}$. This sin/cos embedding is equivariant to the group structure of $\mathbb{Z}/m\mathbb{Z}$ and avoids discontinuities at wrap-around boundaries.

The two feature vectors are projected to the model hidden dimension d by independent projection layers (two-layer MLP for magnitude; affine for modulo):

$$\mathbf{h}_i^{\text{mag}} = \text{MLP}_{\text{mag}}(\mathbf{f}_i^{\text{mag}}), \quad \mathbf{h}_i^{\text{mod}} = W_{\text{mod}} \mathbf{f}_i^{\text{mod}} + \mathbf{b}_{\text{mod}}, \quad \mathbf{h}_i^{\text{mag}}, \mathbf{h}_i^{\text{mod}} \in \mathbb{R}^d.$$

The two streams are fused by FiLM [11]: the modulo embedding generates element-wise scale γ_i and shift β_i :

$$\gamma_i = W_\gamma \mathbf{h}_i^{\text{mod}}, \quad \beta_i = W_\beta \mathbf{h}_i^{\text{mod}}, \quad \mathbf{e}_i = (1 + \gamma_i) \odot \mathbf{h}_i^{\text{mag}} + \beta_i.$$

A ReLU is applied after the modulo projection, and dropout is inserted before FiLM. Standard sinusoidal positional encodings are added to $\mathbf{e}_i$ before the encoder.

The overall architecture is illustrated in Fig. 1.

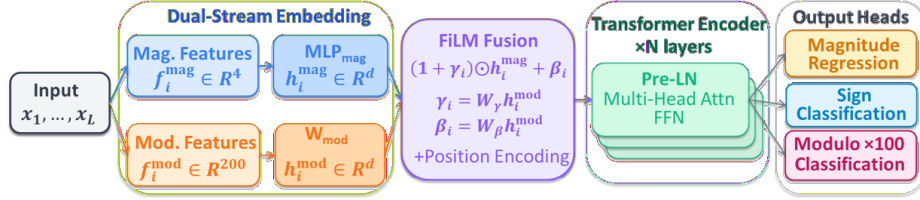


Fig. 1. IntSeqBERT architecture. The Dual-Stream Embedding block projects $\mathbf{f}_i^{\text{mag}} \in \mathbb{R}^4$ and $\mathbf{f}_i^{\text{mod}} \in \mathbb{R}^{200}$ to $\mathbb{R}^d$ via MLP_{mag} and W_{mod} , then fuses them with FiLM: $\mathbf{e}_i = (1 + \gamma_i) \odot \mathbf{h}_i^{\text{mag}} + \beta_i$. Positional encodings are added before the Pre-LN Transformer encoder. Three prediction heads produce $\hat{v}, \log \hat{\sigma}^2 \in \mathbb{R}$ (magnitude regression), $\hat{s} \in \{+, -, 0\}$ (sign classification), and $\hat{r}^{(m)} \in \{0, \dots, m-1\}$ ($100 \times$ modulo classification).

3.3 Encoder, Heads, and Training

The fused sequence $(\mathbf{e}_1, \dots, \mathbf{e}_L)$ is processed by a standard Transformer encoder [18] with Pre-Layer Normalisation [20]. Three model sizes are evaluated (Table 1).

Let $\mathbf{z}_i \in \mathbb{R}^d$ be the encoder output at masked position i . Three prediction heads are applied: (i) **Magnitude head** — a two-layer MLP ($d \rightarrow d \rightarrow 2$, ReLU) outputting $(\mu_i, \log \sigma_i^2)$, predicting $\hat{v}_i = \mu_i$ (a log-scale estimate of v_i ;

Table 1. Model configurations.

Config	Layers	d	Heads	Parameters
Small	6	256	4	6.4M
Middle	8	512	8	29.0M
Large	12	768	12	91.5M

$\sigma_i^2 = \exp(\log \sigma_i^2)$ is used by the Solver); (ii) **Sign head** — $\hat{s}_i = \text{softmax}(W_{\text{sign}} \mathbf{z}_i)$; (iii) **Modulo head** — 100 independent linear classifiers, one per modulus, with total output dimension $\sum_{m=2}^{101} m = 5,150$.

The multi-task loss is $\mathcal{L} = \mathcal{L}_{\text{mag}} + \mathcal{L}_{\text{sign}} + 2\mathcal{L}_{\text{mod}}$. The weight 2 on $\mathcal{L}_{\text{mod}}$ is intentionally set higher to emphasise the newly introduced modulo stream; adaptive weighting was attempted but caused training instability. $\mathcal{L}_{\text{mag}}$ is Huber loss; $\mathcal{L}_{\text{sign}}$ is cross-entropy over three sign classes; $\mathcal{L}_{\text{mod}} = \frac{1}{100} \sum_{m=2}^{101} \frac{1}{\ln m} \mathcal{L}_{\text{CE}}^{(m)}$, where $\mathcal{L}_{\text{CE}}^{(m)}$ is the cross-entropy loss of the m -class classifier for modulus m , normalised by $\ln m$ (the maximum entropy of a uniform distribution over m classes) to compensate for varying class counts. All losses are computed only at masked positions.

3.4 Baselines

Vanilla Transformer maps each integer to a token ID in a vocabulary of 20,003 entries (non-negative values 0–19,999 plus PAD, MASK, UNK); negative integers and values $\geq 20,000$ are replaced by UNK. The same three prediction heads are applied to the token embeddings. The vocabulary size 20,003 was chosen so that memory consumption matches IntSeqBERT under the 8 GB VRAM constraint. The prior benchmark FACT [1] handles values up to several million, presupposing larger compute resources than our setting.

Ablation (magnitude-only) is identical to IntSeqBERT but uses only the magnitude stream; the FiLM module is removed and $\mathbf{e}_i = \mathbf{h}_i^{\text{mag}}$. This isolates the contribution of the modulo stream.

3.5 Solver

The pre-trained model outputs magnitude $(\mu, \log \sigma^2)$, sign, and modulo distributions at masked positions. **Solver** recovers concrete integers from these predictions.

The Solver derives the 3σ interval $[n_{\min}, n_{\max}]$ from the magnitude prediction and dynamically selects one of three modes based on the search width $\Delta n = |n_{\max} - n_{\min}|$ (Table 2).

When the sign prediction is zero, the Solver immediately returns 0. If no valid candidate is found in the search range, the call is recorded as **none**.

Each candidate n is scored by $\text{score}(n) = \alpha_{\text{mag}} + 0.3\alpha_{\text{mod}}$, where $\alpha_{\text{mag}} = -(v_n - \mu_i)^2 / (2\sigma_i^2)$ and $\alpha_{\text{mod}} = \sum_m \ln P(n \bmod m)$. The coefficient 0.3 approximates the proportion of primes between 2 and 101 ($26/100 = 0.26$), compensating for the information overlap among composite moduli that share prime

Table 2. Solver operating modes.

Mode	Condition	Method
Dense	$\Delta n \leq 10^6$	Enumerate all integers in range
Sieve	$10^6 < \Delta n \leq 10^{14}$	CRT beam search anchored on high-confidence moduli
CRT	$\Delta n > 10^{14}$	Sparse CRT beam search to generate large integers directly

factors. The top k candidates are returned and evaluated as Solver Top- k (Section 5.4).

4 Dataset and Experimental Setup

4.1 Dataset

As of January 2026, OEIS contains 391,710 sequences. We exclude sequences with fewer than ten terms and those carrying any of eleven incompatible OEIS tags (out-of-scope representations: `cons/cofr/frac/base/word`; undefined task: `fini/tab1`; quality: `dead/unkn/less/dumb`), yielding 274,705 sequences (70.1%) as our standard dataset (`std`). The intent of this filtering is to narrow the corpus to sequences whose number-theoretic structure can be studied through the integer values themselves: the excluded representations depend strongly on a particular encoding (e.g. decimal digit expansions), and the quality-label tags mark incomplete or deprecated entries. We acknowledge that some of these choices are not strictly necessary: the quality labels `less` and `dumb` are based on subjective criteria, and `tab1` sequences can be encoded as one-dimensional sequences by a standard transformation. These exclusions are a practical decision for constructing a first benchmark; designing a more systematic filtering criterion—for example, classifying sequences by the complexity of their generation rule or by their prediction difficulty—is left for future work.

The corpus is split 8:1:1 (seed 42): 219,765 training / 27,470 validation / 27,470 test sequences.

Each sample corresponds to a sequence prefix of at most $L = 128$ terms (longer sequences are truncated). In the raw test corpus (before truncation), sequence lengths range from 10 to 168 terms (mean 42.5). After truncation, all model inputs satisfy $L \leq 128$, while still ensuring that the Solver receives at least 9 terms of preceding context (Section 5.4).

For scale-stratified evaluation, we define five magnitude buckets by value range:

Limitations of the split. The random split may place related sequences (e.g., Fibonacci and Lucas) in different splits; family-aware splits are left for future work.

Table 3. Magnitude buckets used for scale-stratified evaluation.

Bucket	Value range	Element fraction
Small	$ x < 10^2$	52.7%
Medium	$10^2 \leq x < 10^5$	29.9%
Large	$10^5 \leq x < 10^{20}$	16.2%
Huge	$10^{20} \leq x < 10^{50}$	1.1%
Astronomical	$ x \geq 10^{50}$	0.0%

4.2 Training Configuration

All models share the hyperparameters shown in Table 4. Some OEIS values reach $|x| \approx 10^{210}$, causing FP16 overflow; all computation is therefore conducted in FP32. All results are reported by applying the model trained for the fixed 200 epochs to the test set.

Table 4. Training hyperparameters (common to all models).

Hyperparameter	Value
Epochs	200 (no early stopping)
Batch size	32 (gradient accumulation 2 steps; effective 64)
Learning rate	5×10^{-5}
Warmup fraction	10%
Optimiser	AdamW, weight decay 0.01
Numerical precision	FP32 (AMP disabled)
Mask probability	0.15
GPU	GeForce RTX 3070 Ti (8 GB VRAM) $\times$ 1

4.3 Evaluation Metrics

We evaluate using the following metrics: **Magnitude Accuracy (Mag Acc, $\text{Acc}_{0.5}$)**: fraction of masked positions with $|\hat{v}_i - v_i| < 0.5$ ($\sqrt{10} \approx 3.16 \times$ tolerance in integer scale). This metric does *not* measure exact integer match; it measures whether the magnitude prediction is precise enough to set a realistic search range for the Solver’s Dense and Sieve modes (Section 3.5). Concretely, if the true value is 10^6 , any prediction between roughly 3.16×10^5 and 3.16×10^6 counts as correct. Exact integer match is evaluated separately by Solver Top- k accuracy. **Sign Accuracy (Sign Acc)**: fraction with correct predicted sign class. **Mean Modulo Accuracy (MMA)**: mean classification accuracy across all 100 moduli. **Solver Top- k** : fraction of next-term instances where the true value appears in the Solver’s top- k candidates (Section 5.4). **Normalised Information Gain (NIG)**: $1 - \mathcal{L}_{\text{CE}}^{(m)} / \ln m$ for modulus m ; higher NIG indicates stronger learned periodic structure.

5 Experiments

5.1 Main Results

Table 5 shows test-set performance for all model sizes and variants. IntSeqBERT consistently outperforms both baselines on all scales and metrics. At the Large scale, IntSeqBERT surpasses the Vanilla baseline by +8.9pt in Mag Acc and +4.5pt in MMA. The Ablation model shows the largest MMA drop at Large scale (−15.2pt), directly quantifying the contribution of the arithmetic-periodicity features. Notably, the Ablation maintains non-trivial Mag Acc, but the remaining gap confirms that modulo information contributes meaningfully to magnitude regression as well, while being essential for modulo prediction.

Table 5. Test results. Mag Acc (%), Sign Acc (%), MMA (%). **Bold** indicates the best value within each size group.

Size	Model	Mag Acc	Sign Acc	MMA
Small	IntSeq	94.73	97.78	40.43
Small	Vanilla	85.73	96.91	36.21
Small	Ablation	93.72	97.39	25.97
Middle	IntSeq	95.71	98.34	46.88
Middle	Vanilla	87.37	97.42	42.53
Middle	Ablation	92.45	97.90	31.93
Large	IntSeq	95.85	98.54	50.38
Large	Vanilla	86.97	97.66	45.85
Large	Ablation	89.70	98.29	35.22

Learning curves. Fig. 2 shows validation loss across all variants. Large IntSeqBERT converges from 2.17 to 1.01 over 200 epochs with no overfitting (Train 1.00 / Val 1.01). Vanilla (1.77) and Ablation (1.39) maintain higher final loss.

5.2 Magnitude Prediction

Scale-stratified analysis. Table 6 reports per-bucket MSE on the test set for Large models. The Vanilla model suffers catastrophic degradation in the Large bucket ($\text{MSE} = 2.10$, $13\times$ that of IntSeqBERT), caused by all out-of-vocabulary integers being absorbed into the UNK token. IntSeqBERT achieves the best MSE in all buckets except Small, with particularly pronounced advantage at Medium and above. The Ablation degrades in the Medium bucket without modulo context ($\text{MSE} = 0.116$ vs. IntSeqBERT 0.051), and collapses more severely in the Huge and Astronomical buckets, indicating that FiLM modulation by the modulo stream acts as an arithmetic structural constraint on magnitude estimation for large integers.

Fig. 3 shows predicted vs. true magnitude scatter plots for the three Large model variants. IntSeqBERT achieves the highest coefficient of determination

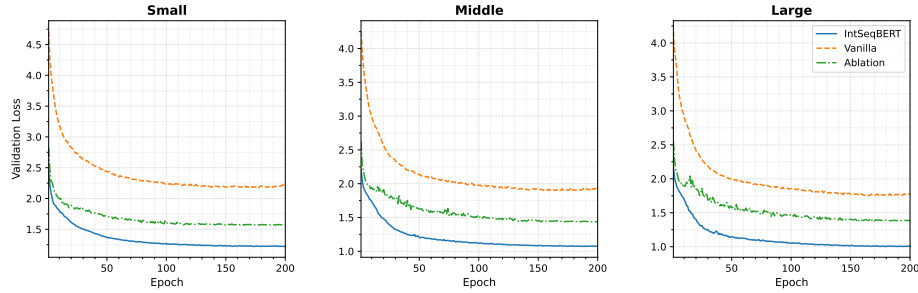


Fig. 2. Validation loss learning curves for all scales (Small / Middle / Large) and all variants. IntSeqBERT (solid blue) consistently achieves lower loss than Vanilla (dashed orange) and Ablation (dash-dot green). At the Large scale, IntSeqBERT converges to Val Loss = 1.01 at epoch 200.

Table 6. Scale-stratified MSE on the test set (Large models). Lower is better.

Bucket	IntSeq	Vanilla	Ablation
Small	0.111	0.138	0.103
Medium	0.051	0.071	0.116
Large	0.162	2.100	0.381
Huge	2.082	22.73	5.021
Astronomical	110.4	840.0	532.6

($R^2 = 0.988$), substantially reducing off-diagonal dispersion in the Large and Huge buckets compared to Vanilla ($R^2 = 0.943$), visually corroborating the MSE results of Table 6.

5.3 Modulo Spectrum Analysis

Fig. 4 shows the NIG spectrum for $m = 2, \dots, 101$ using Large models. IntSeqBERT (solid blue) exceeds both Vanilla and Ablation across the entire spectrum, with a visually clear contrast between prime and composite moduli.

Finding 1: NIG is strongly negatively correlated with Euler’s totient ratio. A Pearson correlation of $r = -0.851$ ($p < 10^{-28}$) is observed between NIG and $\varphi(m)/m = \prod_{p|m} (1 - 1/p)$ (Fig. 5). Composite moduli with many small prime factors achieve higher NIG, consistent with a CRT aggregation effect: when m is a common multiple of smaller moduli $m_1, m_2, \dots$, then $x \bmod m$ encodes the information of all those moduli simultaneously. The highest NIG across all models and scales is achieved at $m = 96 = 2^5 \times 3$ ($\varphi(96)/96 = 1/3$; Large IntSeq: NIG = 0.629, 95% CI [0.622, 0.634]), consistent with $m = 96$ being the largest modulus with totient ratio $1/3$ among $\{12, 24, 48, 72, 96\}$ and thus distinguishing the widest value range. The exception is $m = 2$ (prime, NIG = 0.628, second highest), reflecting the corpus-wide ubiquity of parity in OEIS sequences.

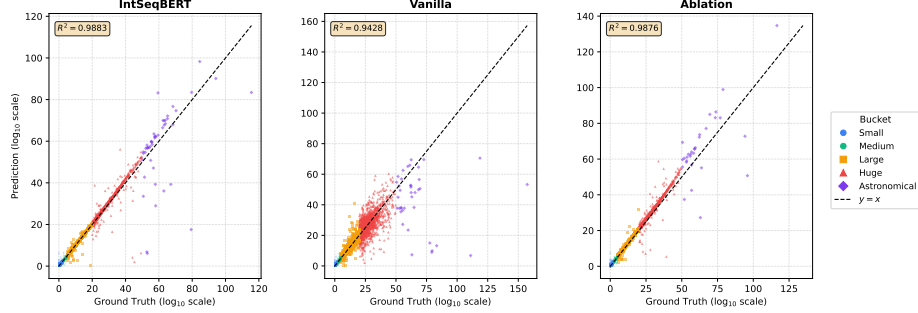


Fig. 3. Predicted vs. true magnitude ($\log_{10}$ scale) for Large models. Points are coloured by bucket (Small=blue circle, Medium=green circle, Large=yellow-orange square, Huge=red triangle, Astronomical=purple diamond). IntSeqBERT achieves $R^2 = 0.988$ vs. Vanilla $R^2 = 0.943$; Vanilla shows pronounced scatter above the Large bucket.

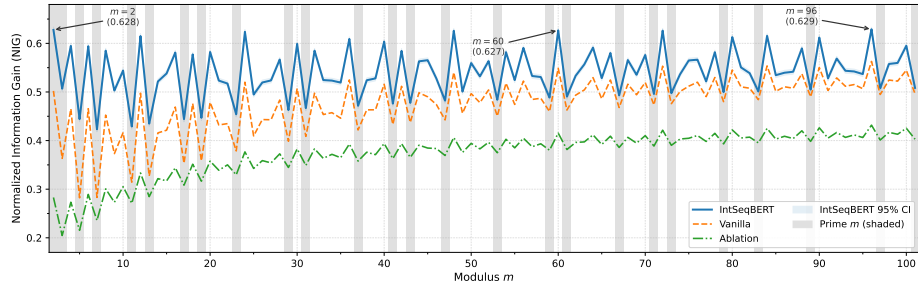


Fig. 4. NIG spectrum for moduli $m = 2, \dots, 101$ (Large models). Grey shading marks prime moduli. The 95% CI for IntSeqBERT (light blue band) is computed by bootstrapping.

While this correlation is an empirical observation rather than a theorem, it carries a design implication for the modulo stream itself: composite moduli reflect constraints from several prime factors and prime powers at once, so the CRT aggregation effect lets a model extract more number-theoretic information from fewer moduli. This gives a guideline for choosing moduli—for example, when extending the spectrum beyond $m = 101$, adding composite numbers with many small prime factors (such as highly composite numbers) is likely to be more effective than simply adding more primes.

Finding 2: Parity (mod 2) accuracy stratifies models. Mod-2 accuracy at the Large scale is 85.65% (IntSeq), 81.40% (Vanilla), and 72.13% (Ablation), making parity the single modulus most sensitive to the modulo stream (−13.5 pt when removed). Table 7 lists representative modulus accuracies.

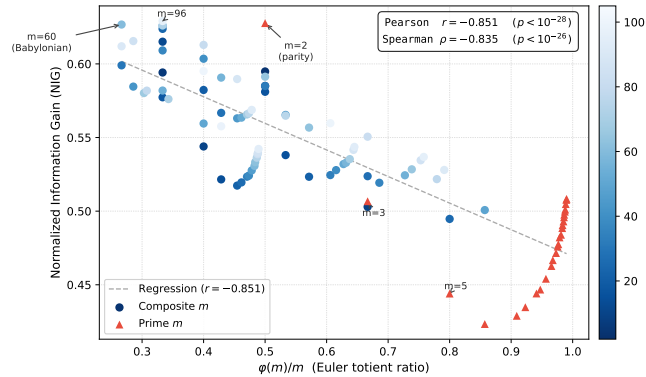


Fig. 5. NIG vs. Euler’s totient ratio $\varphi(m)/m$ (Large IntSeqBERT). Composite moduli (blue circles, shade proportional to m) and prime moduli (red triangles) are shown separately. The regression line (grey dashed) indicates Pearson correlation of $r = -0.851$ ($p < 10^{-28}$). Notable moduli $m = 2$ (parity), $m = 60$ (Babylonian number), and $m = 96$ ($2^5 \times 3$ composite) are annotated.

Table 7. Representative modulus accuracies for Large models (%).

m	IntSeq	Vanilla	Ablation	Interpretation
2	85.65	81.40	72.13	Parity
3	72.62	65.22	53.72	Ternary residue
5	60.37	50.07	42.63	Last decimal digit modulo 5
10	58.38	49.25	39.47	Least significant decimal digit
60	53.97	47.87	35.12	Babylonian number (highly composite)
96	51.82	47.29	34.44	Composite ($2^5 \times 3$)
100	48.51	45.60	33.51	Centesimal residue

5.4 Solver-Based Next-Term Prediction

The Solver reconstructs candidate integers from the model’s magnitude, sign, and modulo predictions and ranks them by likelihood. We evaluate exact-match accuracy on 10,000 test samples, where each sample corresponds to one OEIS sequence and the target is the *last term* (all preceding terms are given as context).

Valid-candidate rate is the fraction of samples for which the Solver returns at least one candidate. Vanilla always returns a candidate from its vocabulary softmax (rate = 100%). IntSeqBERT and Ablation may fail to find a valid candidate within the search range (**none** mode; 13.4% of Large-IntSeq calls), reducing their valid-candidate rate.

Table 8 shows Solver evaluation results. Large-scale IntSeqBERT achieves a Top-1 accuracy of 19.09%, a **7.4-fold** improvement over the Vanilla baseline (2.59%), counting **none** returns as misses.

Accuracy by magnitude bucket. Table 9 compares Solver Top-1 accuracy across buckets for all three Large models. IntSeqBERT achieves 68.34% Top-1 in

Table 8. Solver evaluation: Top-1 and Top-10 exact-match accuracy (%) and valid-candidate rate.

Size	Model	Top-1	Top-10	Sign Acc	Valid rate (%)
Small	IntSeq	14.05	21.00	98.73	90.59
Small	Vanilla	2.43	3.24	92.92	100.0
Small	Ablation	7.42	17.33	98.50	90.17
Middle	IntSeq	17.02	22.62	99.02	86.31
Middle	Vanilla	2.43	3.41	92.71	100.0
Middle	Ablation	9.88	20.52	98.74	90.34
Large	IntSeq	19.09	26.23	99.02	86.64
Large	Vanilla	2.59	3.80	92.05	100.0
Large	Ablation	11.75	21.79	98.94	86.99

the Small bucket, while the Ablation reaches 54.55% and Vanilla only 14.11%. Vanilla collapses to 0% for Medium and above due to the UNK-token degradation of its magnitude predictions, whereas IntSeqBERT retains meaningful accuracy up to the Medium bucket (20.82%).

Table 9. Solver Top-1 / Top-10 accuracy (%) by magnitude bucket (Large models). **Bold:** best per bucket.

Bucket	IntSeqBERT		Vanilla		Ablation	
	Top-1	Top-10	Top-1	Top-10	Top-1	Top-10
Small	68.34	88.50	14.11	20.71	54.55	86.92
Medium	20.82	31.50	0.00	0.00	5.61	18.88
Large	0.31	0.67	0.00	0.00	0.03	0.05
Huge	0.09	0.18	0.00	0.00	0.00	0.00
Astronomical	0.00	0.00	0.00	0.00	0.00	0.00

Mode breakdown (Large IntSeqBERT). Dense mode (24.0% of calls) achieves the highest Top-1 accuracy at 61.06%; Sieve mode (36.7%) achieves 5.36%; the zero-prediction shortcut (2.7%) achieves 89.96%. CRT mode (23.2%) achieves near-zero accuracy (0.09%); 13.4% of calls return no valid candidate. The CRT limitation is discussed in Section 6.4.

6 Analysis and Discussion

6.1 Ablation Study: Contribution of the Modulo Stream

To isolate the contribution of the modulo stream and FiLM fusion, we compare IntSeqBERT with the magnitude-only Ablation model. As seen in Table 5, the modulo stream delivers the largest gains in modulo prediction (MMA +15.2 pt, parity +13.5 pt at Large scale). Its contribution to magnitude prediction is smaller but significant ($\text{Acc}_{0.5}$ +6.2 pt, MSE -0.228), consistent with the intuition that knowing $x_i \bmod m$ for $m \in \{2, \dots, 101\}$ substantially constrains

the set of plausible magnitude values. Solver accuracy improves by +7.3 pt, primarily because more accurate residue information refines scoring in Dense and Sieve modes.

6.2 Scaling Behaviour

Table 10 shows how IntSeqBERT’s metrics improve with model size. Magnitude accuracy improves only modestly from Small to Large (+1.1 pt), whereas modulo accuracy improves substantially (+10.0 pt). This is consistent with the intuition that modular arithmetic is more compositional and thus benefits more from increased representational capacity. Solver Top-1 accuracy follows the modulo trend with consistent improvement (+5.0 pt).

Table 10. IntSeqBERT scaling: test-split metrics (Small / Middle / Large).

Metric	Small	Middle	Large	Δ (S→L)
Mag Acc (%)	94.73	95.71	95.85	+1.12
MMA (%)	40.43	46.88	50.38	+9.95
mod-2 accuracy (%)	81.97	84.50	85.65	+3.68
Solver Top-1 (%)	14.05	17.02	19.09	+5.04
Magnitude MSE	0.228	0.164	0.142	−0.086

The trends in Table 10 are visualised in Fig. 6, which contrasts the differential scaling of modulo and magnitude accuracy across model sizes.

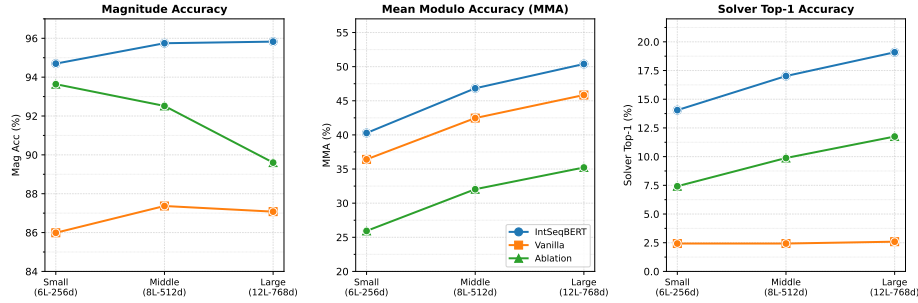


Fig. 6. Scaling behaviour (Small / Middle / Large). Left: Mag Acc improves only +1.1 pt. Centre: MMA for IntSeqBERT improves +10.1 pt (40.3%→50.4%). Right: Solver Top-1 improves +5.0 pt, with a growing gap over Vanilla.

6.3 Alternative Representations: Digit-Based Tokenisation

A natural alternative to our handcrafted features is digit-based tokenisation, treating each decimal digit as a separate token, possibly with digit-by-digit gen-

eration on the output side. We have not made an experimental comparison with digit-level representations in this paper. What our representation targets is not the pattern of decimal digits but the number-theoretic structure of the integer itself: the modulo stream supplies the residues of each value under 100 different moduli—information about modular arithmetic—to the model directly, whereas the same information is only indirectly recoverable from a digit sequence. Digit-by-digit generation is a reasonable alternative on the output side, but it changes the problem setting from predicting an integer as a single value to the sequential prediction of a digit string. A controlled comparison between the two settings is an interesting direction for future work.

6.4 Limitations

Difficulty with large integers. Solver accuracy collapses to near zero for $|x| \geq 10^{20}$ (Huge and Astronomical buckets): CRT-based reconstruction requires *exact* residues for multiple moduli simultaneously, and errors are frequent enough at $\text{MMA} \approx 50\%$ to cause failure (Top-1 = 0.09% in CRT mode). Approximately 13% of IntSeqBERT Solver calls return no valid candidate (**none** mode), primarily due to CRT failures; approximate CRT or relaxed residue constraints are a natural remedy.

Dataset bias. OEIS sequences are predominantly non-negative (**nonn**). The high sign accuracy (98.54%) may partially reflect this bias. Similarly, sequences with linear or sub-linear growth remain the majority even after our filtering, and the high overall magnitude accuracy partially reflects this growth bias; the contribution of our representation shows up within this bias as the large accuracy gains in the Large and higher magnitude buckets (Tables 6 and 9) and the effectiveness of the modulo spectrum (Section 5.3). Astronomical-bucket metrics are based on few samples and should be interpreted with caution.

Compute constraints. All experiments were conducted on a single consumer GPU (GeForce RTX 3070 Ti, 8 GB VRAM), which constrains model size and vocabulary capacity. Due to this budget, all reported runs use a single random seed (42), and we do not provide multi-seed variance estimates. The Vanilla baseline’s vocabulary (20,003) is smaller than that of FACT [1], which handles values up to several million using larger compute resources. Consequently, our Vanilla results represent a FACT-equivalent architecture under our resource constraints rather than a direct reproduction of FACT’s reported numbers.

7 Conclusion

We have presented **IntSeqBERT**, a dual-stream Transformer encoder that fuses a continuous log-scale magnitude embedding with sin/cos modulo embeddings via FiLM, jointly trained on 219,765 OEIS sequences. Experiments (Section 5) demonstrate that the dual-stream design substantially outperforms both a standard tokenised Transformer and a magnitude-only ablation across all scales and metrics.

Our results also carry a broader message. Even with number-theoretic structure built into the representation, the absolute success rate of next-term prediction remains modest (Top-1: 19.09%), and a purely tokenised Transformer is far weaker still (2.59%)—confirming that neural networks struggle with direct prediction of OEIS terms. IntSeqBERT itself is a neural-symbolic design: the network predicts magnitude, sign, and residues separately, and the symbolic CRT solver supplies the exact number-theoretic constraints that neural networks struggle to learn end-to-end. We therefore read our results both as evidence of the limits of pure end-to-end learning on this task and as motivation for hybrid symbolic–neural approaches to computer mathematics.

Future work includes combining magnitude uncertainty estimation with approximate CRT to improve large-integer prediction; extending the modulo stream beyond $m = 101$; evaluating on the remaining FACT benchmark tasks—sequence classification, sequence similarity, next sequence-part prediction, and unmasking [1]—to enable direct comparison against that benchmark; constructing family-aware dataset splits and introducing sequence-synthesis data augmentation to improve generalisation in sparse magnitude buckets; scaling to larger models pre-trained on the full OEIS corpus; and exploring downstream applications such as conjecture generation, where pretrained IntSeqBERT embeddings could serve as a rich feature backbone.

Acknowledgments. This work was supported by JSPS KAKENHI Grant Number JP24K14897.

References

1. Belčák, P., Kastrati, A., Schenker, F., Wattenhofer, R.: FACT: Learning governing abstractions behind integer sequences. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) *Advances in Neural Information Processing Systems 35* (NeurIPS 2022), Datasets and Benchmarks Track. pp. 17968–17980. Curran Associates, Inc. (2022)
2. Birch, B.J., Swinnerton-Dyer, H.P.F.: Notes on elliptic curves. II. *Journal für die reine und angewandte Mathematik* **218**, 79–108 (1965). <https://doi.org/10.1515/crll.1965.218.79>
3. Candelas, P., de la Ossa, X.C., Green, P.S., Parkes, L.: A pair of Calabi-Yau manifolds as an exactly soluble superconformal theory. *Nuclear Physics B* **359**(1), 21–74 (1991). [https://doi.org/10.1016/0550-3213\(91\)90292-6](https://doi.org/10.1016/0550-3213(91)90292-6)
4. Conway, J.H., Norton, S.P.: Monstrous moonshine. *Bulletin of the London Mathematical Society* **11**(3), 308–339 (1979). <https://doi.org/10.1112/blms/11.3.308>
5. Gauss, C.F.: *Disquisitiones Arithmeticae*. Gerhard Fleischer, Leipzig (1801), english translation by Arthur A. Clarke, Yale University Press, 1966
6. Gauthier, T., Olšák, M., Urban, J.: Alien coding. *International Journal of Approximate Reasoning* **162**, 109009 (2023). <https://doi.org/10.1016/j.ijar.2023.109009>
7. Gauthier, T., Urban, J.: Learning program synthesis for integer sequences from scratch. In: *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence (AAAI 2023)*. vol. 37, pp. 7670–7677. AAAI Press (2023). <https://doi.org/10.1609/aaai.v37i6.25930>

8. Gauthier, T., Urban, J.: Learning conjecturing from scratch. arXiv preprint arXiv:2503.01389 (2025). <https://doi.org/10.48550/arXiv.2503.01389>
9. Hubert, T., Mehta, R., Sartran, L., Horváth, M.Z., Žužić, G., Wieser, E., Huang, A., Schrittwieser, J., Schroecker, Y., Masoom, H., Bertolli, O., Zahavy, T., Mandhane, A., Yung, J., Beloshapka, I., Ibarz, B., Veeriah, V., Yu, L., Nash, O., Lezeau, P., Mercuri, S., Sönne, C., Mehta, B., Davies, A., Zheng, D., Pedregosa, F., Li, Y., von Glehn, I., Rowland, M., Albanie, S., Velingker, A., Schmitt, S., Lockhart, E., Hughes, E., Michalewski, H., Sonnerat, N., Hassabis, D., Kohli, P., Silver, D.: Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature* **651**(8106), 607–613 (2026). <https://doi.org/10.1038/s41586-025-09833-y>, published online 12 November 2025; IMO silver-medal result announced July 2024
10. Kaggle: Integer sequence learning. <https://www.kaggle.com/competitions/integer-sequence-learning> (2016), accessed June 2026
11. Perez, E., Strub, F., de Vries, H., Dumoulin, V., Courville, A.: FiLM: Visual reasoning with a general conditioning layer. In: Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI 2018). pp. 3942–3951. AAAI Press (2018). <https://doi.org/10.1609/aaai.v32i1.11671>
12. Raayoni, G., Gottlieb, S., Manor, Y., Pisha, G., Harris, Y., Mendlovic, U., Haviv, D., Hadad, Y., Kaminer, I.: Generating conjectures on fundamental constants with the Ramanujan Machine. *Nature* **590**(7844), 67–73 (2021). <https://doi.org/10.1038/s41586-021-03229-4>
13. Rahaman, N., Baratin, A., Arpit, D., Dräxler, F., Lin, M., Hamprecht, F.A., Bengio, Y., Courville, A.: On the spectral bias of neural networks. In: Proceedings of the 36th International Conference on Machine Learning (ICML 2019). Proceedings of Machine Learning Research, vol. 97, pp. 5301–5310. PMLR (2019)
14. Romera-Paredes, B., Barekatin, M., Novikov, A., Balog, M., Kumar, M.P., Dupont, E., Ruiz, F.J.R., Ellenberg, J.S., Wang, P., Fawzi, O., Kohli, P., Fawzi, A.: Mathematical discoveries from program search with large language models. *Nature* **625**(7995), 468–475 (2024). <https://doi.org/10.1038/s41586-023-06924-6>
15. Sloane, N.J.A.: The On-Line Encyclopedia of Integer Sequences. Published electronically at <https://oeis.org> (1996), accessed 2026
16. Trinh, T.H., Wu, Y., Le, Q.V., He, H., Luong, T.: Solving olympiad geometry without human demonstrations. *Nature* **625**(7995), 476–482 (2024). <https://doi.org/10.1038/s41586-023-06747-5>
17. Urban, J.: 130k lines of formal topology in two weeks: Simple and cheap autoformalization for everyone? arXiv preprint arXiv:2601.03298 (2026). <https://doi.org/10.48550/arXiv.2601.03298>
18. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I.: Attention is all you need. In: Guyon, I., von Luxburg, U., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) *Advances in Neural Information Processing Systems 30* (NeurIPS 2017). pp. 5998–6008. Curran Associates, Inc. (2017)
19. Wiles, A.: Modular elliptic curves and Fermat’s last theorem. *Annals of Mathematics* **141**(3), 443–551 (1995). <https://doi.org/10.2307/2118559>
20. Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., Liu, T.: On layer normalization in the transformer architecture. In: Proceedings of the 37th International Conference on Machine Learning (ICML 2020). Proceedings of Machine Learning Research, vol. 119, pp. 10524–10533. PMLR (2020)